

A-TO-Z MASTER ENGINEERING BLUEPRINT

WhatsApp API Gateway & AI Chat Support Platform

Exhaustive technical documentation covering every implementation detail, rationale behind technology choices versus industry alternatives, the PostgreSQL SKIP LOCKED outbox concurrency engine, multi-tier anti-ban mathematics, and complete API specifications.

Architecture:	Transactional Outbox + Async Event-Driven
Concurrency Engine:	PostgreSQL FOR UPDATE SKIP LOCKED
WhatsApp Engine:	Baileys WebSocket Driver (Noise Protocol)
Vector Storage:	PostgreSQL + pgvector (768-D dense embeddings)
AI Intelligence:	Google Gemini 1.5 Flash via LangChain JS
Target Environment:	Production Docker / Kubernetes / Node.js 20+

1. Executive Summary & Core Topology

The WhatsApp Gateway serves as an autonomous, self-hosted communication bridge that provides enterprise microservices, CRMs, and automated AI agents with reliable, ban-safe programmatic access to WhatsApp.

Unlike conventional wrapper libraries that crash on redeployments or get numbers banned within minutes due to uncontrolled bursts, this platform implements a hardened, multi-tier architecture:

- **Transactional Outbox:** Decouples synchronous HTTP requests from asynchronous WhatsApp socket transmissions with 100% ACID consistency.
- **Anti-Ban Engine:** Enforces mathematical rate limits (4 msgs/min strict cap, 5s inter-message hardware gap, 10s same-recipient cooldown, 6 new contacts/hr velocity cap, and 45–90s burst pauses).
- **RAG AI Support Bot:** Retrieves semantic documentation chunks from PostgreSQL using vector cosine similarity, passing context to Google Gemini 1.5 Flash to automatically answer customer queries.

2. Database Architecture & Data Models (PostgreSQL)

Why PostgreSQL 16 Was Chosen Over MongoDB, MySQL, and DynamoDB

Requirement	PostgreSQL 16 (Chosen)	MongoDB	MySQL 8	DynamoDB
ACID Outbox Transaction	Atomic Commit Writes message record and queue job in single SQL transaction.	High Overhead Multi-doc transactions are heavy and lock collections.	Supported (InnoDB), but lacks native partial indexing for queue states.	Unsupported No multi-table atomic rollback without costly transactions API.
Vector Storage (RAG)	pgvector Extension 768-d vector embeddings in the same DB as chats and keys.	Requires Atlas Vector Search (paid proprietary cloud lock-in).	No native vector support; requires external vector plugin.	Requires AWS OpenSearch integration (\$\$\$).
Queue Concurrency	SKIP LOCKED Non-blocking distributed dequeuing.	Relies on optimistic locks, causing high write conflicts.	Supported, but table locking semantics are more aggressive.	Requires external AWS SQS queue service.

Schema Entity Overview

- sessions** : Tracks multi-device pairing state (**starting** , **qr** , **connected** , **logged_out**), phone numbers, and last activity timestamps.
- auth_states** : Stores cryptographic Signal keys, pre-keys, and app-state sync credentials directly in PostgreSQL JSONB, enabling instant container redeployment without physical QR re-scanning.
- messages** : Complete audit log of all inbound and outbound messages, statuses (**queued** , **sent** , **delivered** , **read** , **failed**), and idempotency keys.
- outbox_jobs** : The execution queue containing message payloads, retry counters, locking metadata (**lockedAt** , **lockedBy**), and cooldown timestamps (**nextRetryAt**).
- knowledge_documents** & **knowledge_chunks** : The RAG repository storing document metadata alongside 768-dimensional vector embeddings.
- webhooks** & **webhook_deliveries** : Subscribed HTTP callback endpoints and immutable dispatch audit logs.

3. The Outbox Concurrency Engine: Why SKIP LOCKED Over BullMQ & RabbitMQ

Detailed Comparative Analysis

Evaluation Vector	PostgreSQL SKIP LOCKED (Chosen)	BullMQ (Redis)	RabbitMQ (AMQP)
The Dual-Write Bug	Mathematically Impossible Message record and queue job are committed in the same database transaction.	High Hazard If Redis flushes or times out after DB insert, the message is orphaned forever.	High Hazard Network partitioned broker causes database vs. queue state inconsistency.
Strict FIFO per Session	Native SQL Guarantee <code>WHERE sessionId = :id ORDER BY id ASC LIMIT 1.</code>	Complex; requires BullMQ Pro grouped keys or dynamic per-number queues.	Complex; requires single-active consumer per queue.
Dynamic Cooldown Rescheduling	Zero Memory Footprint <code>nextRetryAt = NOW() + cooldown.</code> Job sleeps on disk until exact millisecond.	Expensive; keeps delayed jobs in Redis RAM ZSETs.	Cumbersome; requires Dead Letter Exchanges (DLX) and TTL routing.
Infrastructure Footprint	1 Single Engine Zero extra servers to manage, monitor, or pay for.	2 Engines: PostgreSQL + Redis Cluster + Sentinel.	2 Engines: PostgreSQL + RabbitMQ Broker + Erlang VM.

The Core Dequeue Query

```
UPDATE "outbox_jobs"
SET
  status = 'processing',
  "lockedAt" = :now,
  "lockedBy" = :workerId
WHERE id = (
  SELECT id FROM "outbox_jobs"
  WHERE status = 'queued'
  AND "sessionId" = :sessionId
  AND "nextRetryAt" <= :now
  ORDER BY id ASC
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
RETURNING *;
```

4. WhatsApp Driver Layer: Baileys vs. Alternatives

Why Baileys Was Chosen

- **Zero Per-Message Fees:** Unlike Meta's Cloud API (which charges \$0.03–\$0.08 per 24-hour conversation window and requires corporate Meta Business verification), Baileys connects via native WhatsApp multi-device WebSockets with zero API charges.
- **Lightweight Resource Consumption:** Unlike Puppeteer or Selenium (which spawn headless Chromium instances consuming 800MB–1.5GB of RAM per session), Baileys operates purely in JavaScript memory (<40MB RAM per session) using WhatsApp's binary Protobuf protocol.
- **PostgreSQL Signal Key Adapter:** Stores all identity keys, signed keys, and pre-keys in PostgreSQL rather than local disk files, preventing file-lock corruption across clustered Docker containers.

5. The Multi-Tier Anti-Ban Engine & Behavioral Pacing

WhatsApp uses automated behavioral heuristics to detect automated accounts. The gateway implements an enterprise **Anti-Ban Engine** (`src/services/antiBan.service.js`) enforcing 6 strict mathematical barriers:

Rule 1: Rolling 60-Second Window (Max 4 msgs/min in Strict Mode): An array of timestamps in the last 60 seconds is inspected. The 5th message is deferred until the oldest message falls out of the window.

Rule 2: Hardware Minimum Gap (5.0s): Enforces a mandatory cooldown of 5,000ms between any two consecutive dispatches, preventing burst spikes.

Rule 3: Same-Recipient Cooldown (10.0s): Prevents spamming the same recipient. Messages to the same contact are paced at least 10 seconds apart.

Rule 4: Cold Contact Velocity Limiter (Max 6 New Contacts / Hour): Prevents accounts from mass-messaging unknown contacts. The 7th new contact in an hour is deferred to the next hour.

Rule 5: Natural Burst Pauses ("Coffee Break"): After 10 continuous messages, injects a natural human pause of 45 to 90 seconds.

Rule 6: Daily Warmup Schedule (Day 1: 25 msgs/day): New numbers start at 25 msgs/day, increasing by 20% daily at midnight up to 300 msgs/day.

6. The RAG Knowledge Pipeline & AI Support Bot

Google Gemini 1.5 Flash + LangChain JS

- **Model Choice:** Google Gemini 1.5 Flash provides an industry-leading free tier (15 requests/min, 1,500 requests/day, 1M tokens/min) with ultra-low latency (<600ms Time-to-First-Token) and a 1-million-token context window.
- **Embeddings:** Google's `text-embedding-004` produces dense 768-dimensional vector representations.
- **In-Database Vector Cosine Similarity:**

```
SELECT c.id, c.content, d.title, cosine_similarity(c.embedding, :queryVec) as similarity
FROM knowledge_chunks c
JOIN knowledge_documents d ON c."documentId" = d.id
ORDER BY similarity DESC LIMIT 4;
```

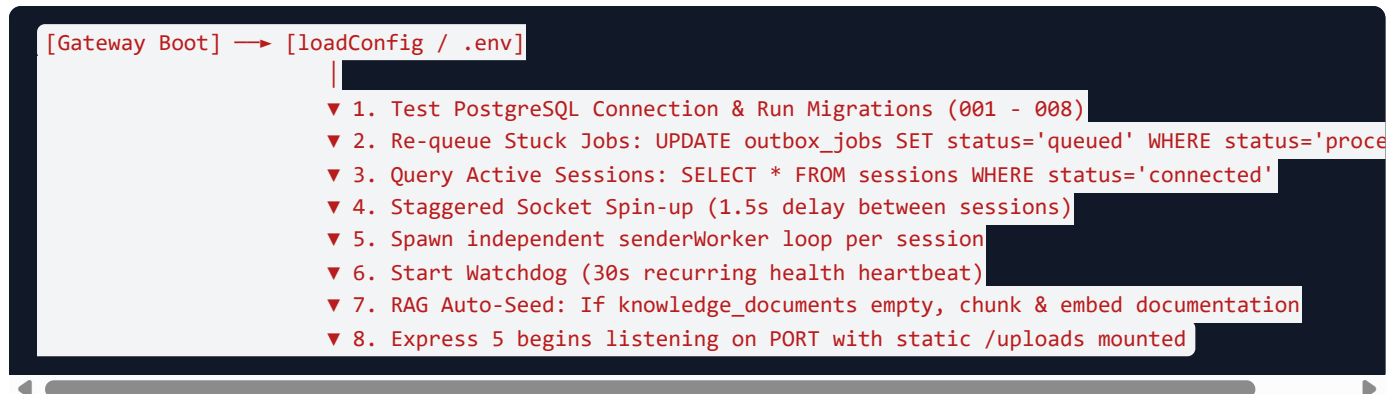
- **Production Guardrails:**
 - Self-message filter: Never replies to our own messages (`fromMe === true`).
 - Human operator pause: If a human agent replies to a chat from the phone, the bot automatically pauses for 5 minutes.
 - Rate limiting: Maximum 5 bot replies per minute per chat JID.

7. Networking, Webhooks & Communications: Why Not Socket.io?

- **WhatsApp Protocol Incompatibility:** WhatsApp Web communicates via binary WebSockets using the Noise Protocol. Socket.io cannot talk to WhatsApp's native socket protocol.
- **Webhooks Are the Industry Standard for B2B:** External microservices, CRMs, and serverless functions (AWS Lambda, Cloud Run) cannot maintain persistent Socket.io connections. They require standard signed HTTP POST webhooks (`POST /webhook`).
- **HMAC-SHA256 Signing:** All webhook deliveries include an `X-Signature` header computed as: `sha256 = HMAC_SHA256(rawBody, webhookSecret)`, ensuring payloads cannot be forged or tampered with in transit.

8. End-to-End System Flows & Step-by-Step Working Mechanism

Step-by-Step Mechanism 1: Server Bootup & Auto-Restoration



- Configuration Loading:** Loads environment variables, validates database credentials, anti-ban profile (`strict` / `moderate` / `relaxed`), and AI settings.
- Database Migration Check:** Verifies all 8 migrations (sessions, auth_states, messages, outbox_jobs, webhooks, webhook_deliveries, api_keys, knowledge_base) are applied.
- Crash Recovery & Lock Reclamation:** Any jobs left in `status: 'processing'` from a previous server crash are immediately reset to `'queued'` with `lockedBy = null`.
- Session Reconnection:** Queries all previously connected sessions from `sessions` table and loads cryptographic Signal keys from `auth_states`.
- Staggered Socket Startup:** Connects Baileys instances staggered 1.5 seconds apart to avoid simultaneous connection bursts to WhatsApp servers.
- Worker Spawning:** Launches a dedicated `senderWorker` instance for each active WhatsApp session.
- Watchdog Activation:** Starts a 30-second recurring watchdog (`src/workers/watchdog.js`) monitoring socket readyState.
- RAG Auto-Seeding:** Automatically inspects `knowledge_documents`; if empty, ingests the system documentation, generates 768-D vectors via Google `text-embedding-004`, and indexes them in PostgreSQL.

Step-by-Step Mechanism 2: Multi-Device QR Pairing & Signal Handshake



- Session Registration:** Client posts `POST /api/v1/sessions` with `{ "id": "sales-1", "name": "Sales Line" }`.
- Noise Protocol Handshake:** Baileys opens a WebSocket with WhatsApp servers and emits the pairing string via the `'qr'` event.
- QR Code Rendering:** Client requests `GET /api/v1/sessions/:id/qr?format=png`, rendered dynamically via `qrcode`.

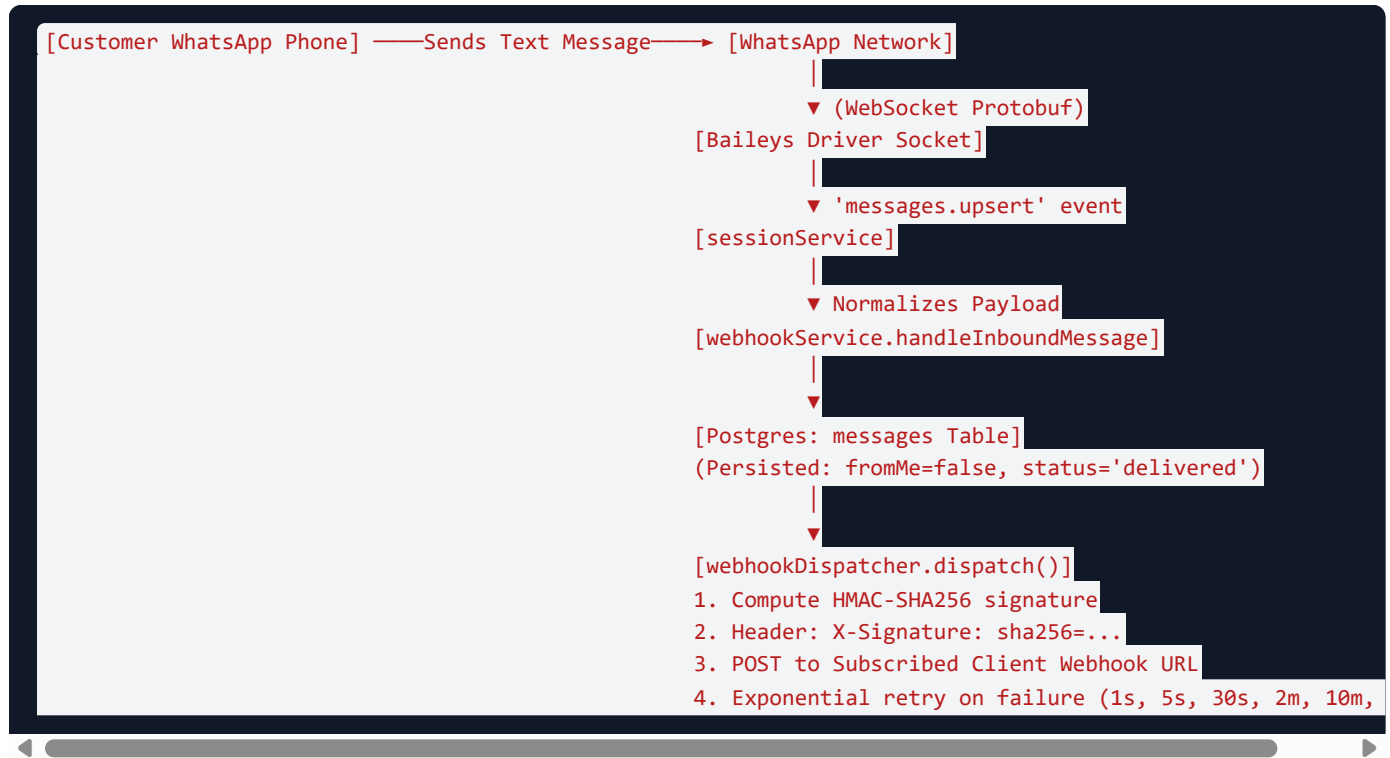
4. **Physical Device Scan:** Admin scans the QR code from their phone (**WhatsApp > Linked Devices > Link a Device**).
5. **Cryptographic Signal Exchange:** WhatsApp and Baileys exchange Curve25519 identity keys and pre-keys.
6. **PostgreSQL Storage:** Our custom auth adapter serializes all session credentials into `auth_states`.
7. **Connected State:** Baileys emits `'connected'` with the authenticated JID; gateway transitions session status to `'connected'` and spawns the `senderWorker`.

Step-by-Step Mechanism 3: Outbound Message Sending with Anti-Ban Pacing



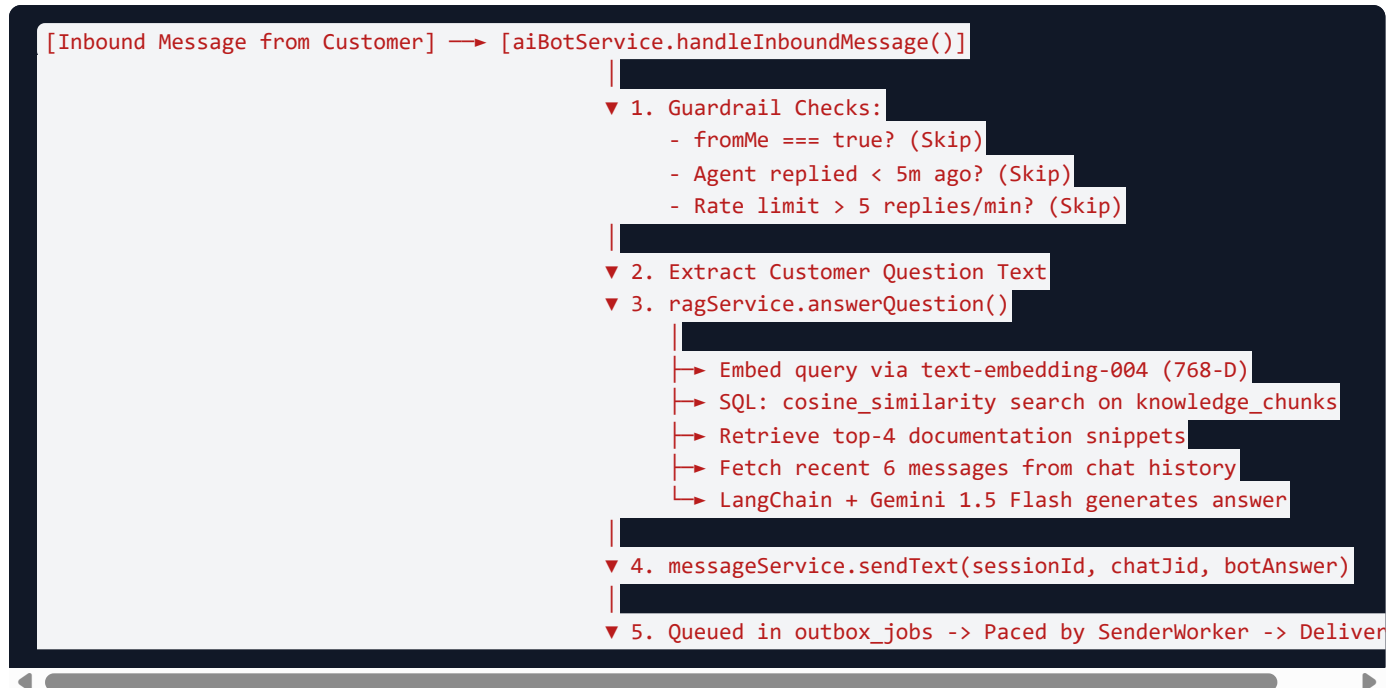
- Client Request:** External service sends `POST /api/v1/sessions/:id/messages/text`.
- API Key Verification:** Token hash verified against `api_keys` for `send` permission.
- Atomic Outbox Commit:** Single SQL transaction inserts into `messages` and `outbox_jobs`. Client instantly receives `HTTP 202 Accepted`.
- Single-Stream Dequeue:** `senderWorker` executes `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1`.
- Anti-Ban Gatekeeper:** Checks 6 hard rules: 4 msgs/min window, 5s hardware gap, 10s same-recipient gap, 6 cold contacts/hr, 45-90s burst rest, daily warmup cap.
- Reschedule on Limit:** If restricted, the job is unlocked and rescheduled in PostgreSQL for `NOW() + waitMs`. Worker memory remains free.
- Human Presence Emulation:** Emits `available` and `composing` (phone shows "typing..."), waiting 35-75ms per character.
- Socket Dispatch & Jitter:** Message transmitted via Baileys WebSocket; status marked `'sent'`; worker sleeps 4-9s random jitter before next job.

Step-by-Step Mechanism 4: Inbound Message Handling & Signed Webhook Delivery



1. **Customer Dispatch:** Customer sends a message from their WhatsApp app.
2. **Signal Decryption:** Baileys decrypts the Protobuf frame and emits `messages.upsert`.
3. **Database Insertion:** Message persisted in `messages` with `fromMe: false` and `status: 'delivered'`.
4. **Cryptographic Signing:** Webhook dispatcher computes `HMAC-SHA256(payload, secret)` and sets `X-Signature`.
5. **HTTP POST Dispatch:** Webhook delivered to client endpoint; on failure, retried automatically with exponential backoff.

Step-by-Step Mechanism 5: Autonomous AI Support Bot & RAG Knowledge Loop



1. **Inbound Trigger:** `webhookService.handleInboundMessage` forwards incoming customer message to `aiBotService`.
2. **Safety Guardrails:** Verifies `fromMe === false`, confirms no human agent replied in the last 5 minutes, and checks sliding rate limit (≤ 5 replies/min).
3. **Dense Vector Embedding:** Generates a 768-dimensional normalized float embedding via Google's `text-embedding-004`.
4. **PostgreSQL Cosine Similarity Search:** Executes `cosine_similarity(embedding, queryVec)` to find the top 4 most relevant documentation snippets.
5. **Multi-Turn Context:** Retrieves recent 6 messages from `messages` table to provide conversational continuity.
6. **AI Synthesis:** Google Gemini 1.5 Flash synthesizes an accurate, grounded, helpful answer via LangChain JS.
7. **Outbox Pacing:** The reply is enqueued into `outbox_jobs`, paced through human typing emulation, and sent back to the customer's WhatsApp chat.

Step-by-Step Mechanism 6: Microphone Voice Recording & Push-To-Talk (PTT)



1. **Microphone Stream:** React dashboard accesses microphone via `navigator.mediaDevices.getUserMedia({ audio: true })`.
2. **HTML5 MediaRecorder:** Captures audio chunks with live elapsed timer and visual recording pulse.
3. **Instant In-Browser Preview:** Assembles `audio/webm` Blob and generates an instant preview via `<audio controls>`.
4. **Multipart Upload:** Dispatches multipart form data to `POST /api/v1/sessions/:id/messages/voice/upload`.
5. **Disk Streaming:** Multer stores audio in `uploads/` with unique timestamp UUID.
6. **PTT Dispatch:** Sender worker executes Baileys `sendMessage(jid, { audio: { url }, ptt: true })`.
7. **Native WhatsApp Voice Note:** Delivered to customer's phone as a true WhatsApp voice note with audio waveform scrubber and green microphone icon.

9. Complete REST & Webhook API Reference

A. Session Management

Method	Endpoint	Description
POST	/api/v1/sessions	Creates a WhatsApp session. Body: <code>{"id": "sales-1", "name": "Sales Line"}</code> .
GET	/api/v1/sessions/:id/qr	Renders pairing QR code as image (<code>format=png</code>) or JSON (<code>format=json</code>).
DELETE	/api/v1/sessions/:id	Logs out and destroys stored session credentials.
POST	/api/v1/sessions/:id/simulate/incoming	Injects a mock customer message for sandbox testing with zero ban risk.

B. Message Sending Endpoints

```
# Send Text Message
POST /api/v1/sessions/sales-1/messages/text
Authorization: Bearer <API_KEY>
Content-Type: application/json

{
  "to": "919876543210",
  "text": "Hello! Your support ticket #4102 has been resolved.",
  "idempotencyKey": "ticket-4102-resolved"
}

# Upload and Send Microphone Voice Note (PTT)
POST /api/v1/sessions/sales-1/messages/voice/upload
Authorization: Bearer <API_KEY>
Content-Type: multipart/form-data

file: [binary audio: .webm / .ogg / .mp3]
to: "919876543210"
idempotencyKey: "voice-101"

# Upload and Send Image
POST /api/v1/sessions/sales-1/messages/image/upload
Authorization: Bearer <API_KEY>
Content-Type: multipart/form-data

file: [binary image: .png / .jpg]
to: "919876543210"
caption: "Product catalog"

# Upload and Send Document / PDF
POST /api/v1/sessions/sales-1/messages/document/upload
Authorization: Bearer <API_KEY>
Content-Type: multipart/form-data

file: [binary document: .pdf / .docx]
to: "919876543210"
```

C. RAG Knowledge Base Endpoints

```
# Ingest Document (.pdf, .docx, .txt, .md)
```

```
POST /api/v1/rag/documents/upload
```

```
Authorization: Bearer <API_KEY>
```

```
Content-Type: multipart/form-data
```

```
file: [binary file]
```

```
title: "Shipping Policy 2026"
```

```
# Query Knowledge Base Directly
```

```
POST /api/v1/rag/query
```

```
Authorization: Bearer <API_KEY>
```

```
Content-Type: application/json
```

```
{  
  "query": "How do I return a damaged product?",  
  "topK": 3  
}
```

```
# Configure Gemini Key & Bot Status
```

```
POST /api/v1/rag/settings
```

```
Authorization: Bearer <API_KEY>
```

```
Content-Type: application/json
```

```
{  
  "apiKey": "AIzaSy...",  
  "botEnabled": true,  
  "sessionId": "global"  
}
```

10. Operational Runbook, Resilience & Production Deployment

Component	Configuration Key	Production Value	Rationale
Provider Engine	WA_PROVIDER	baileys (or mock for testing)	Baileys for live multi-device WhatsApp, Mock for zero-ban unit testing.
Anti-Ban Pacing	ANTI_BAN_MODE	strict	4 msgs/min, 5s hardware gap, 45-90s burst pauses for maximum account longevity.
AI Model	GEMINI_MODEL	gemini-1.5-flash	Sub-second inference, generous free tier quotas, 1M token context window.
Vector Embeddings	GEMINI_EMBEDDING_MODEL	text-embedding-004	768-dimensional normalized dense vectors for semantic knowledge matching.
Queue Worker	OUTBOX_POLL_INTERVAL_MS	1000	Low-overhead poll interval using SKIP LOCKED without lock contention.
Crash Recovery	LOCK_TIMEOUT_MS	300000 (5 min)	Automatic reclamation of stalled jobs if a worker node crashes mid-send.